

Deep Learning

Session 3

Agenda:

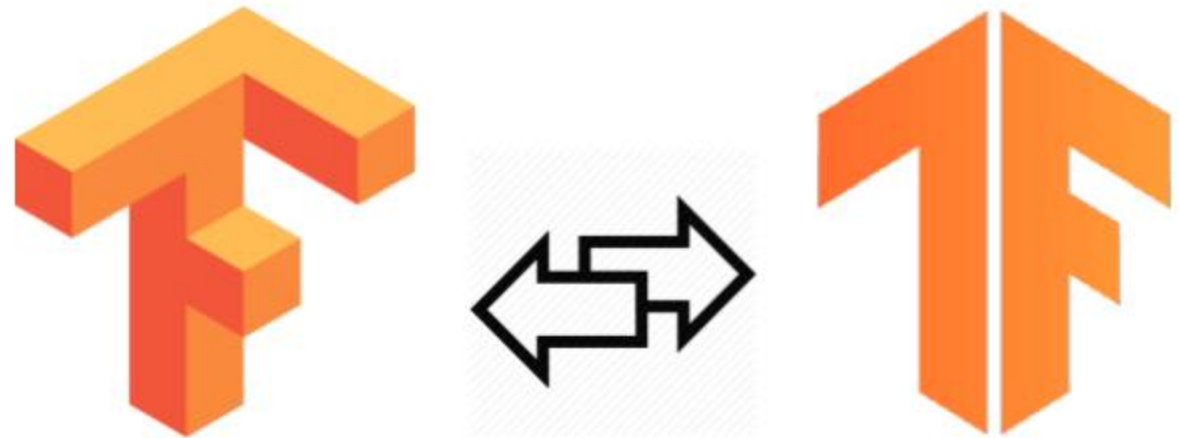
1	TensorFlow
2	Introduction to Transfer Learning
3	Hyperparameter Tuning
4	Early Stopping and Model Checkpointing

TensorFlow

TensorFlow

TensorFlow v1 vs TensorFlow v2

- TensorFlow has evolved significantly since its initial release, and the transition from **TensorFlow v1** to **TensorFlow v2** brought about several important changes, improving usability and making the framework more efficient and intuitive.



TensorFlow

1. TensorFlow v1 (Legacy Version)

- **TensorFlow v1** was the first version of TensorFlow and was initially released in 2015. It was powerful, but not very user-friendly. Here are some key characteristics:

TensorFlow

1. TensorFlow v1 (Legacy Version)

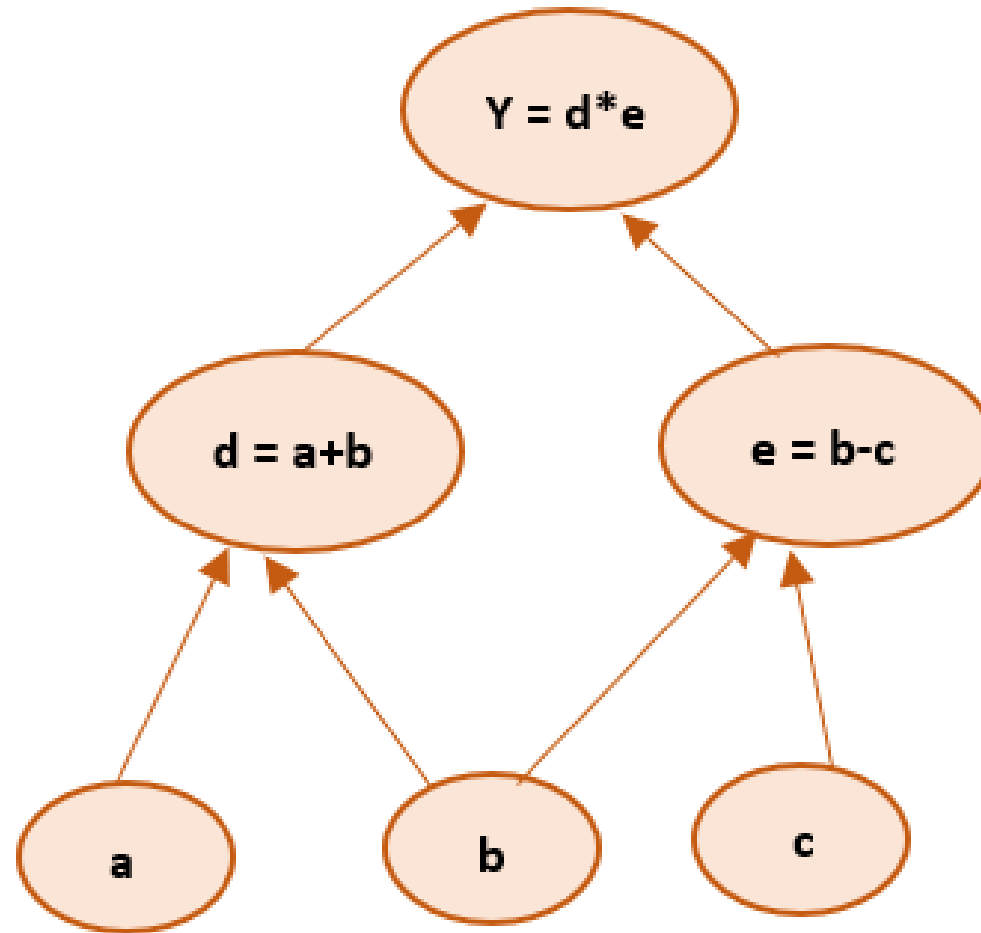
Static Graphs

- TensorFlow v1 operates with a **static computation graph**. This means that the computation graph (representing the neural network model) must be fully defined before it can be executed. You first build the graph, then run it with a session.

TensorFlow

1. TensorFlow v1 (Legacy Version)

Static Graphs



TensorFlow

1. TensorFlow v1 (Legacy Version)

Placeholders and Sessions

- Users need to explicitly define placeholders (placeholders are like variables for input) and use **sessions** to run the graph. This made TensorFlow v1 more complex and less intuitive compared to modern frameworks like PyTorch.

TensorFlow

1. TensorFlow v1 (Legacy Version)

Eager Execution Disabled by Default

- TensorFlow v1 did not support **eager execution**, meaning that computations were deferred until the session was executed. This sometimes made debugging and dynamic operations more challenging.

TensorFlow

2. TensorFlow v2 (Modern Version)

- **TensorFlow v2**, released in 2019, was a significant improvement over TensorFlow v1. It made the library more user-friendly, concise, and Pythonic. Here are the key features:

TensorFlow

2. TensorFlow v2 (Modern Version)

Eager Execution by Default

- TensorFlow v2 enables **eager execution** by default, meaning that operations are executed immediately, much like standard Python code. This allows for more intuitive code and easier debugging.

TensorFlow

2. TensorFlow v2 (Modern Version)

Simplified API

- TensorFlow v2 has a much more simplified API. Many functions, such as `tf.placeholder`, were removed, and APIs were streamlined. This improved code readability and made TensorFlow more beginner-friendly.

TensorFlow

2. TensorFlow v2 (Modern Version)

No Sessions Required

- Unlike TensorFlow v1, there is no need to use **sessions** to execute a computation graph. TensorFlow v2 operates more like regular Python code.

TensorFlow

2. TensorFlow v2 (Modern Version)

Support for Keras

- TensorFlow v2 integrates deeply with **Keras**, a high-level API, making model building, training, and evaluation much more straightforward. Keras became the default API for defining models in TensorFlow.

TensorFlow

2. TensorFlow v2 (Modern Version)

Backward Compatibility

- TensorFlow v2 provides backward compatibility for TensorFlow v1 code using the `tf.compat.v1` module. This ensures that legacy TensorFlow v1 code can still be executed in TensorFlow v2, making the transition easier for developers.

TensorFlow

Why Migrate to TensorFlow v2 ?

- **Easier to Use:** TensorFlow v2 is much more intuitive and feels like regular Python code, making it easier to use for both beginners and advanced users.
- **Improved Debugging:** With eager execution, errors are caught immediately, making debugging much faster and more effective.
- **Seamless Integration with Keras:** TensorFlow v2 simplifies model building with the Keras API, allowing for faster prototyping and deployment of models.

TensorFlow

Migration from TensorFlow v1 to v2

- TensorFlow v2 provides several tools to help developers migrate their code from v1 to v2, including the `tf_upgrade_v2` script, which automatically updates old TensorFlow v1 code to be compatible with v2.

For example : `tensorflow_upgrade_v2 --infile old_script.py --outfile new_script_v2.py`

This tool automatically replaces deprecated functions, updates code structure, and makes it easier to adapt existing codebases to TensorFlow v2.

TensorFlow

Summary of TensorFlow v1 vs v2

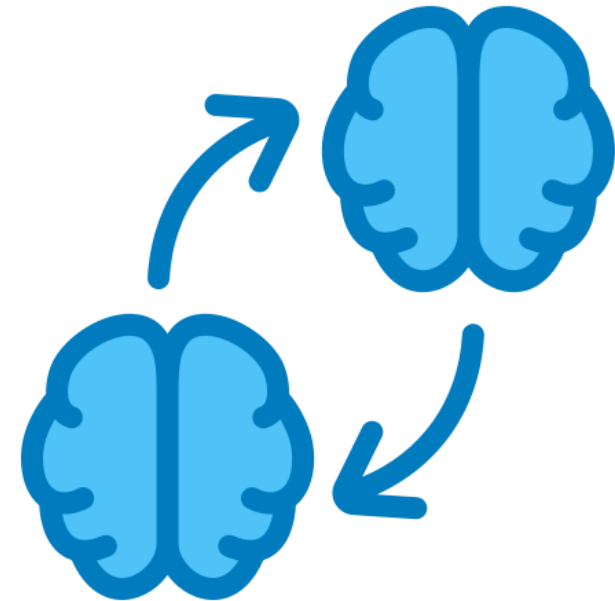
- **TensorFlow v1:** More powerful but complex, requiring static graphs, placeholders, and sessions. Debugging was harder, and code was less Pythonic.
- **TensorFlow v2:** Streamlined and user-friendly, with eager execution, simplified APIs, and seamless Keras integration. It's now the default for TensorFlow users and developers.
- **TensorFlow v2** is a more advanced, robust, and flexible version of TensorFlow, making it easier to build and deploy machine learning models, and is highly recommended for most use cases today.

Introduction to Transfer Learning

Introduction to Transfer Learning

What is Transfer Learning?

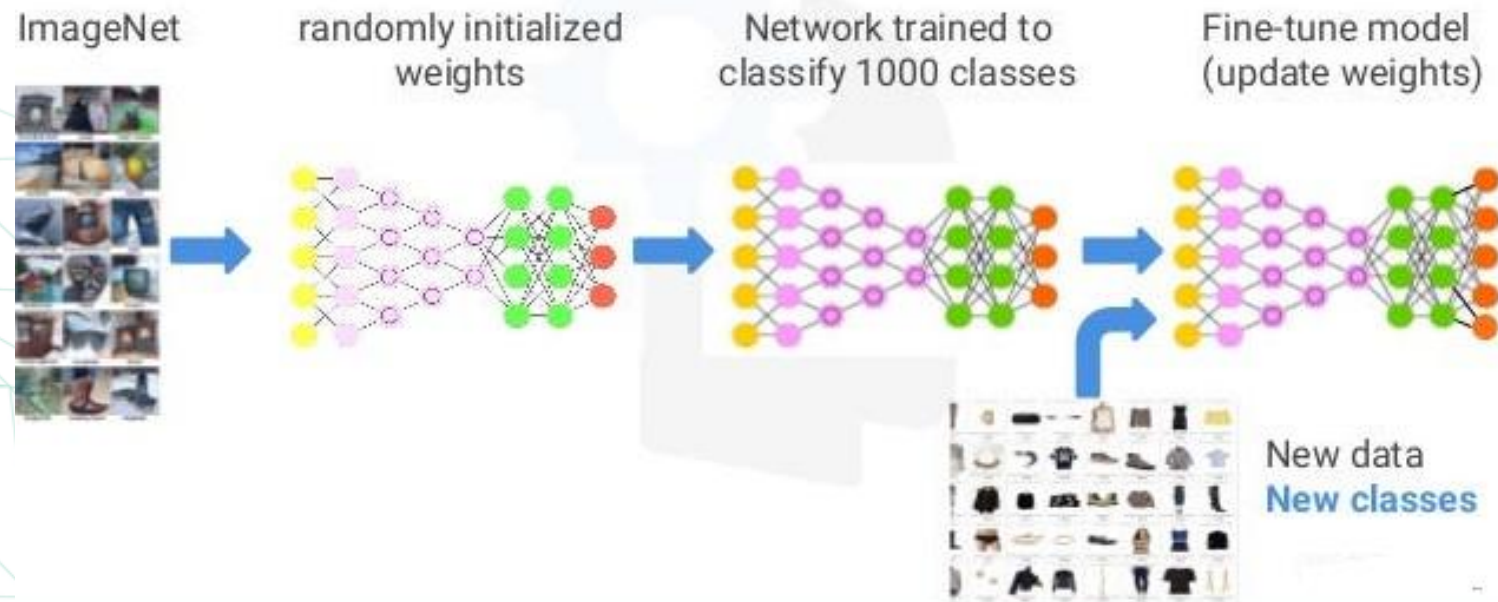
- Transfer Learning is a technique where a model trained on one task is reused for a different, but related task. Instead of training a model from scratch, we can leverage pre-trained models that have already learned useful features on large datasets (e.g., ImageNet) and fine-tune them for specific tasks.



Introduction to Transfer Learning

What is Transfer Learning?

Transfer Learning

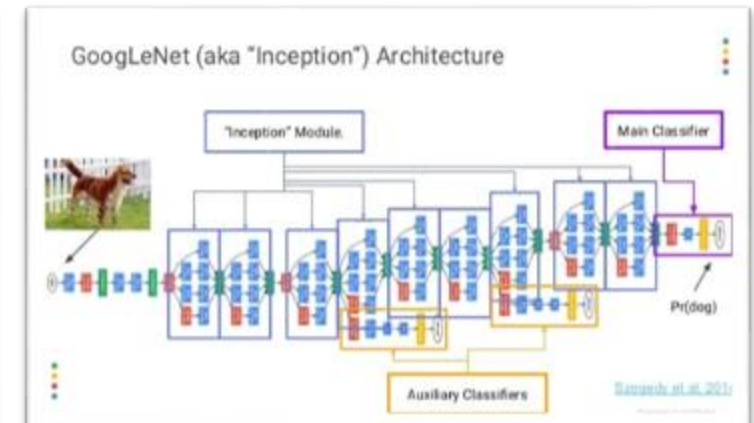
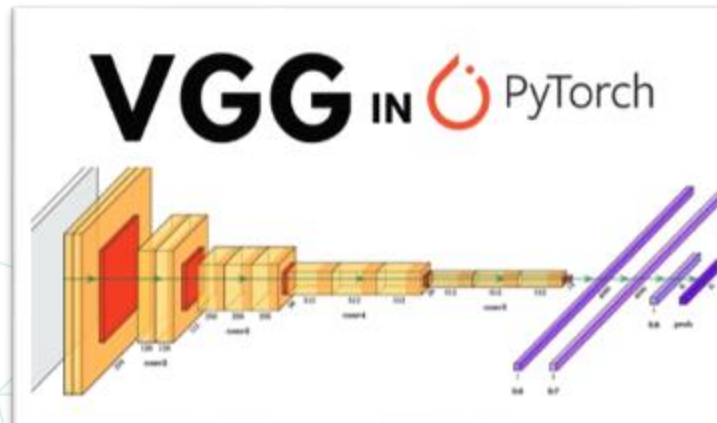
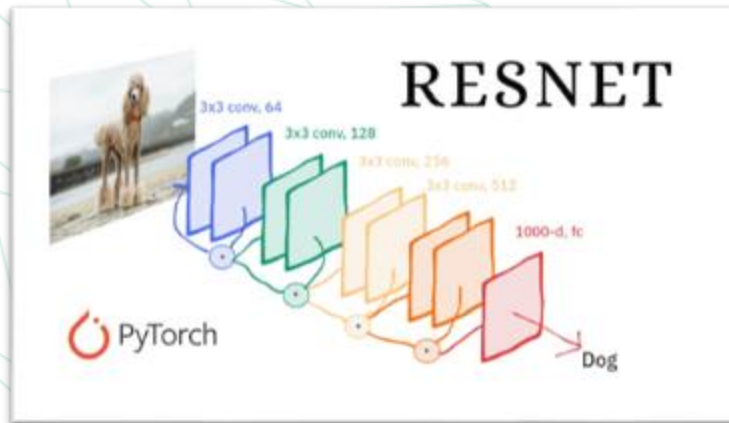


Introduction to Transfer Learning

How Transfer Learning Works ?

1. Use a Pre-trained Model

- Models like **ResNet**, **VGG**, or **Inception** are trained on large datasets and have already learned to detect general features (e.g., edges, textures).



Introduction to Transfer Learning

How Transfer Learning Works ?

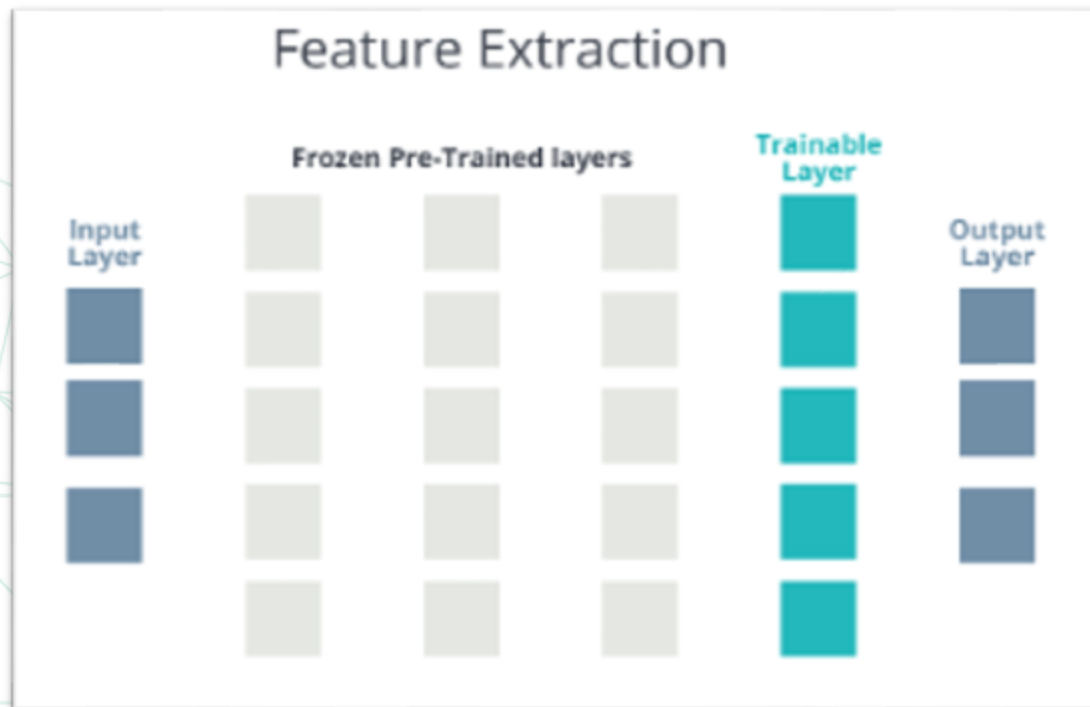
2. Freeze Early Layers

- In Transfer Learning, you typically freeze the earlier layers of the pre-trained model since they contain general features useful for many tasks.

Introduction to Transfer Learning

How Transfer Learning Works ?

2. Freeze Early Layers



Introduction to Transfer Learning

How Transfer Learning Works ?

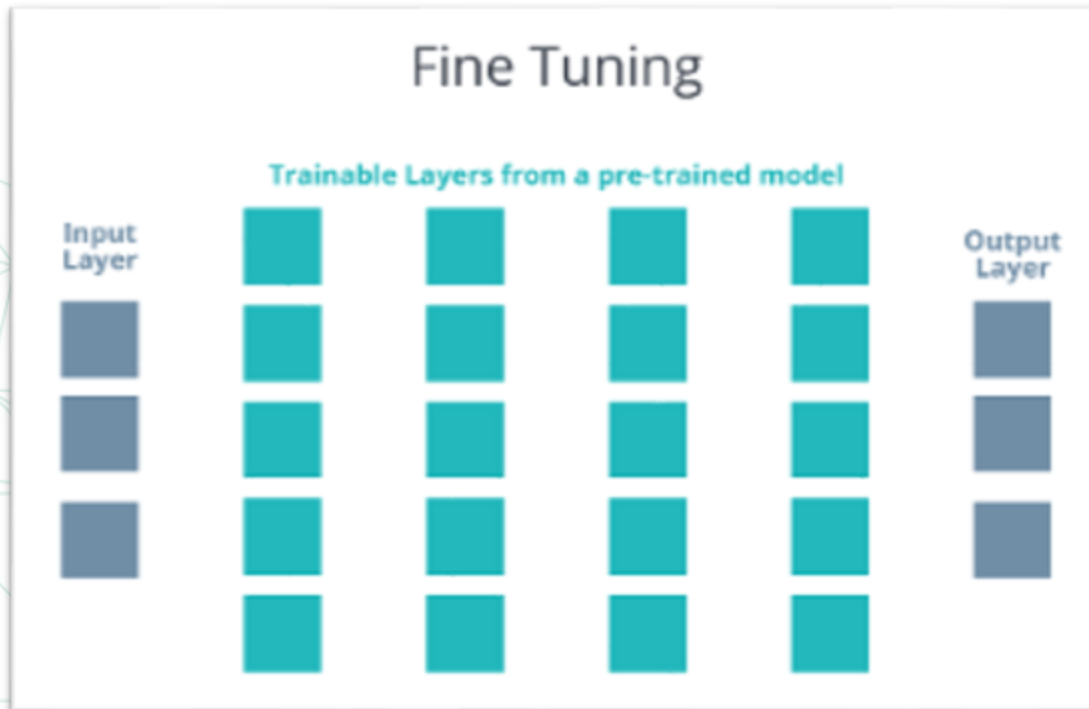
3. Fine-tune the Last Layers

- Replace the final layers (classifier layers) with new layers suited for your specific task and fine-tune these layers on your dataset.

Introduction to Transfer Learning

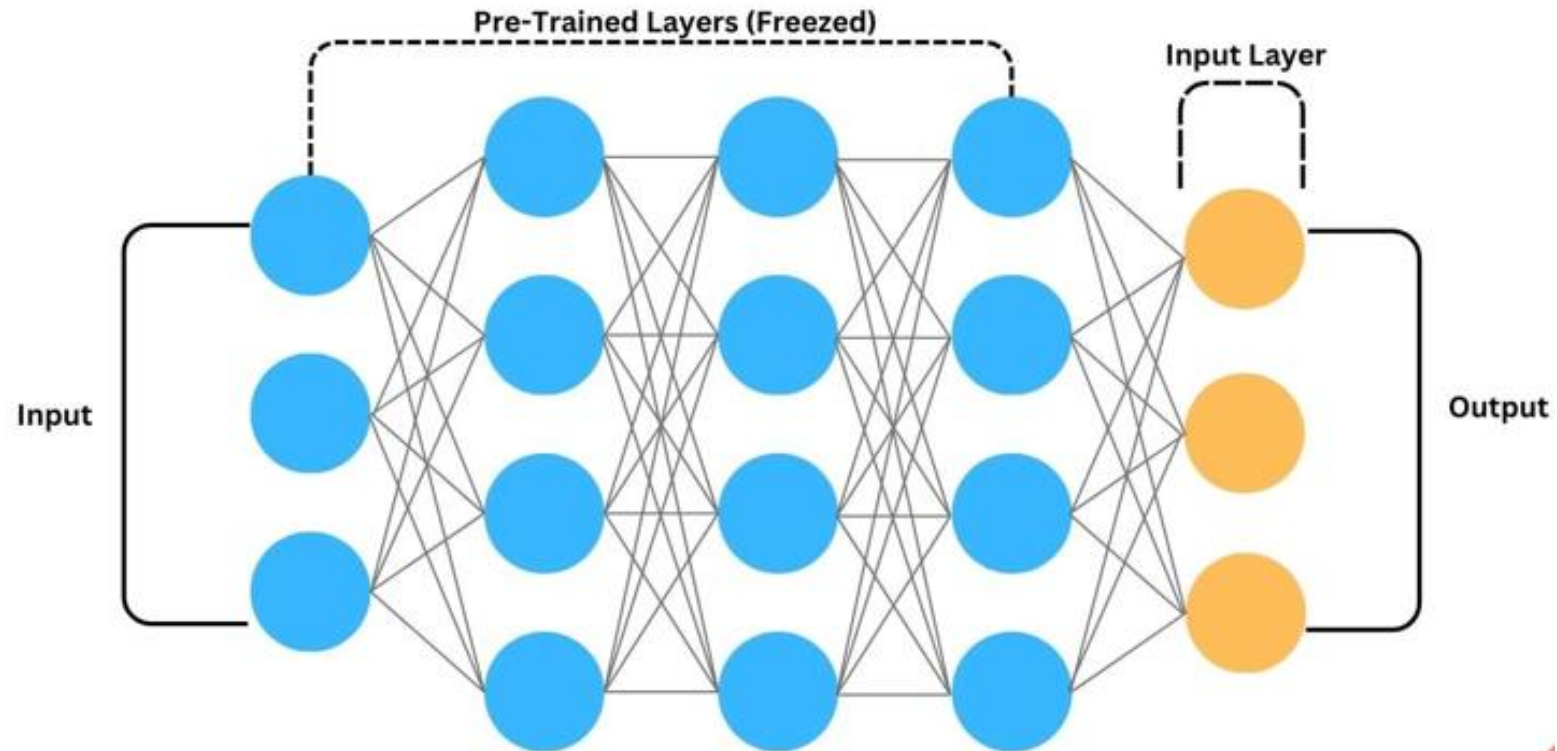
How Transfer Learning Works ?

3. Fine-tune the Last Layers



Introduction to Transfer Learning

How Transfer Learning Works ?



Introduction to Transfer Learning

Benefits of Transfer Learning

Saves Time

- By leveraging pre-trained models, you avoid training from scratch, significantly reducing training time.



Introduction to Transfer Learning

Benefits of Transfer Learning

Requires Less Data

- Transfer learning is especially useful when you have a small dataset. Pre-trained models generalize well to new tasks, even with limited data.



Introduction to Transfer Learning

Benefits of Transfer Learning

Better Performance

- Often leads to improved performance since the model has already learned useful features from a large dataset.

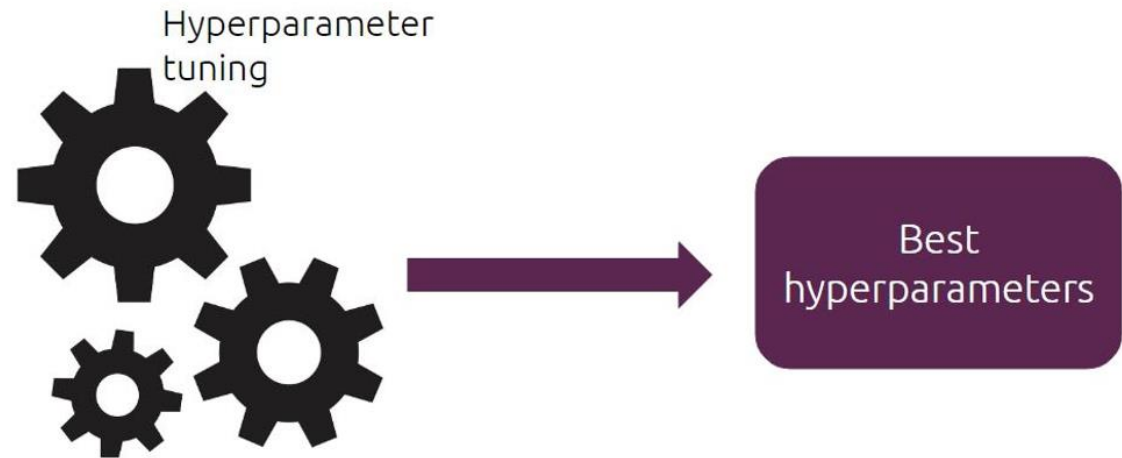


Hyperparameter Tuning

Hyperparameter Tuning

What is Hyperparameter Tuning?

- Hyperparameters are settings used to control the learning process (e.g., learning rate, batch size, number of layers, and number of neurons). Tuning hyperparameters involves finding the best combination of values to improve the model's performance.

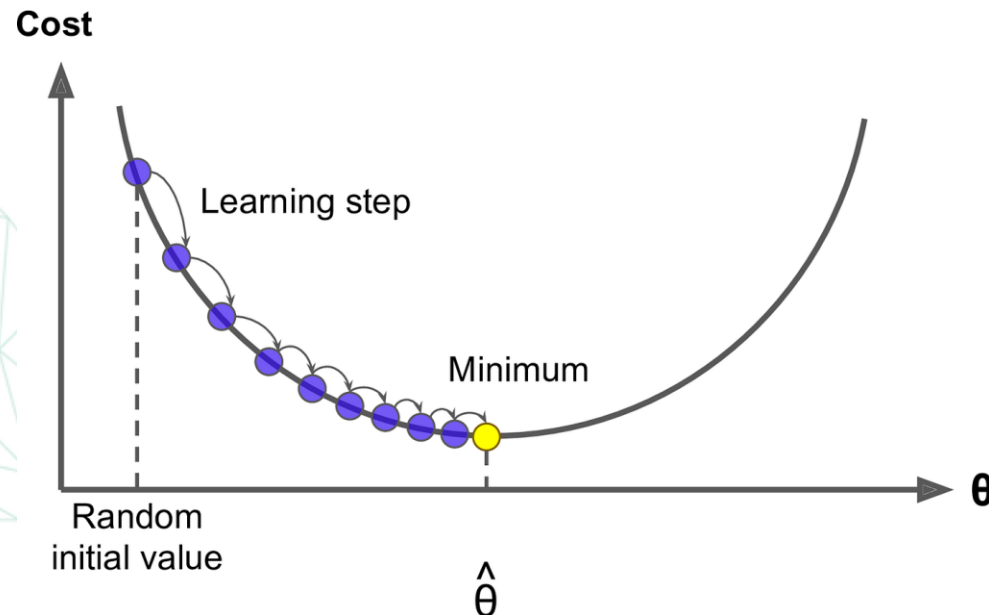


Hyperparameter Tuning

Common Hyperparameters to Tune

1. Learning Rate

- Controls how big the steps are during optimization. Too high can cause the model to overshoot the optimal weights, while too low can lead to slow convergence.

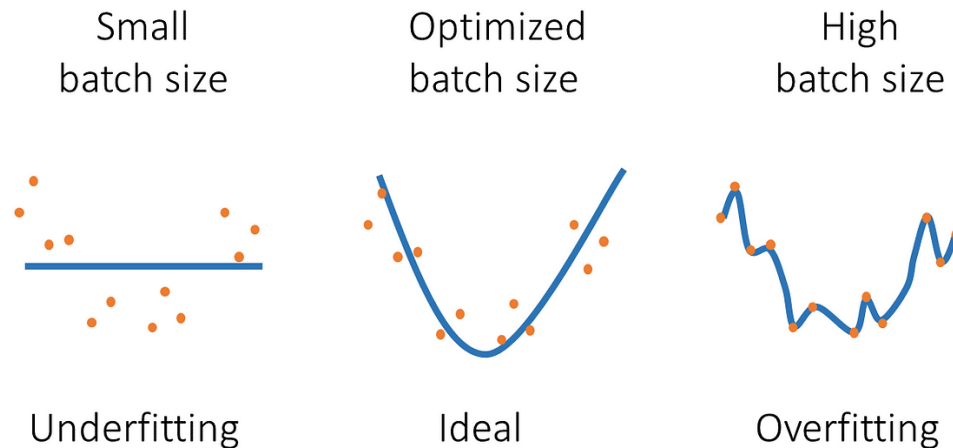


Hyperparameter Tuning

Common Hyperparameters to Tune

2. Batch Size

- Determines how many samples are processed before the model's weights are updated. Smaller batch sizes can lead to noisier gradient updates, but sometimes generalize better.

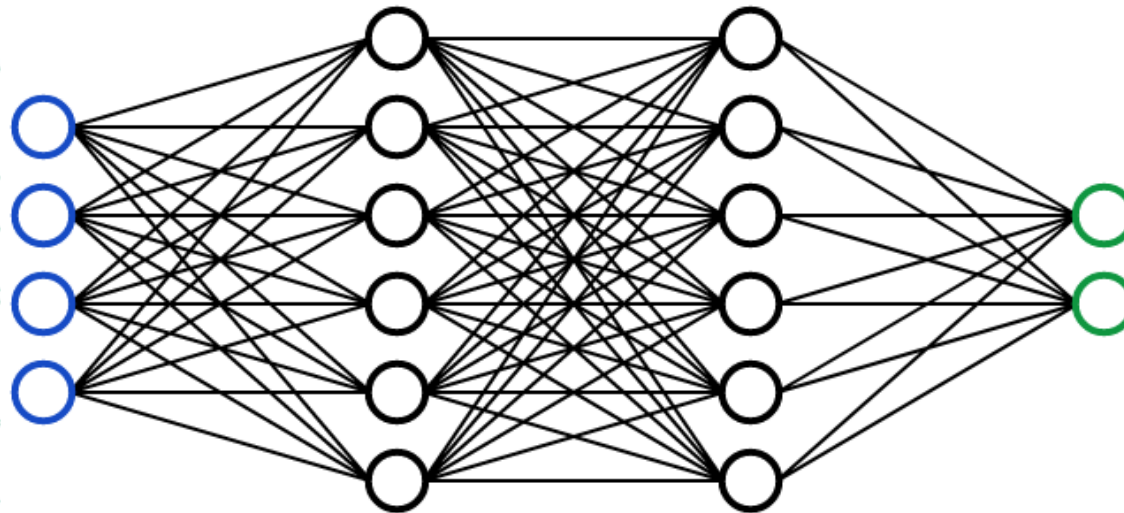


Hyperparameter Tuning

Common Hyperparameters to Tune

3. Number of Neurons/Layers

- More layers or neurons in hidden layers can capture more complex features, but they also increase the risk of overfitting.



Hyperparameter Tuning

Common Hyperparameters to Tune

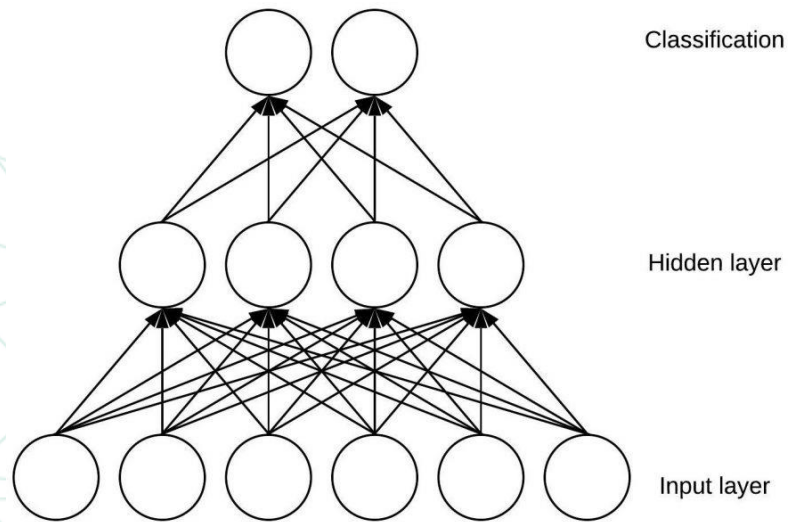
4. Dropout Rate

- Percentage of neurons to drop during training to prevent overfitting

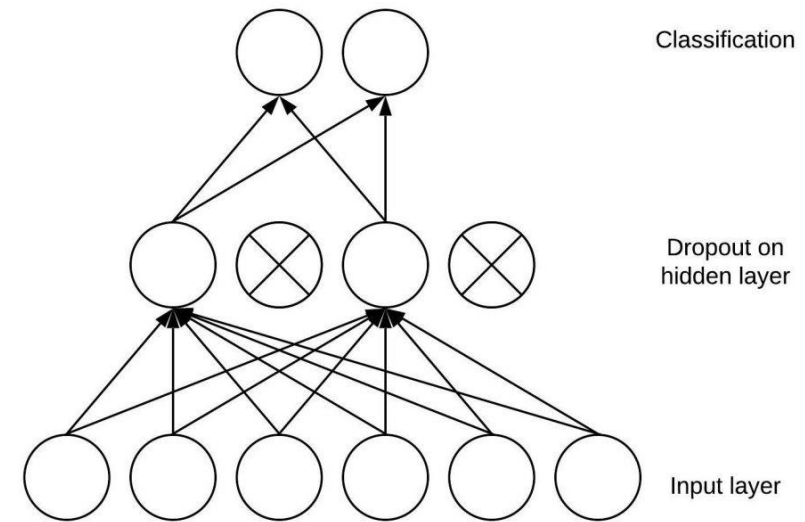
Hyperparameter Tuning

Common Hyperparameters to Tune

4. Dropout Rate



Without Dropout

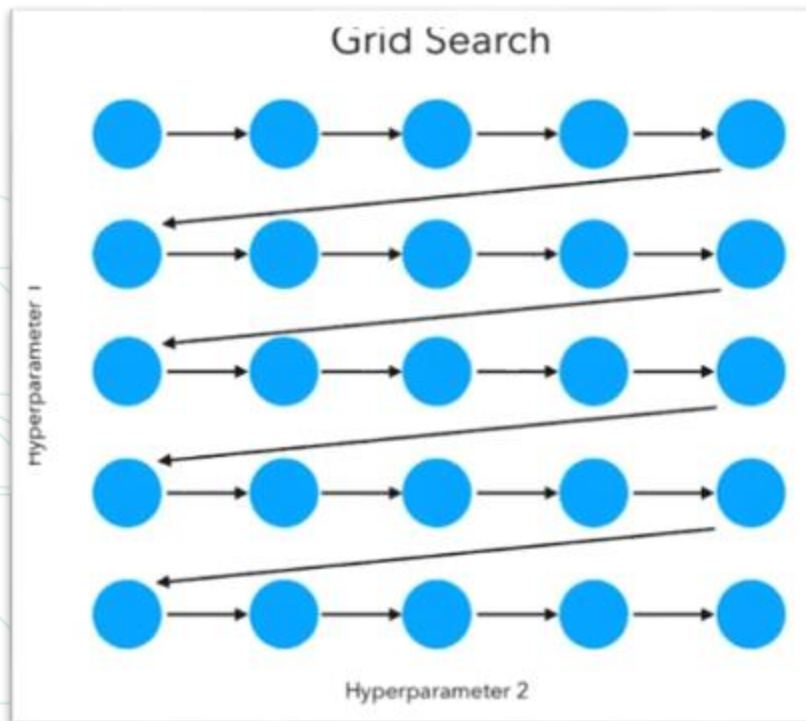


With Dropout

Hyperparameter Tuning

Hyperparameter Tuning Techniques

Grid Search



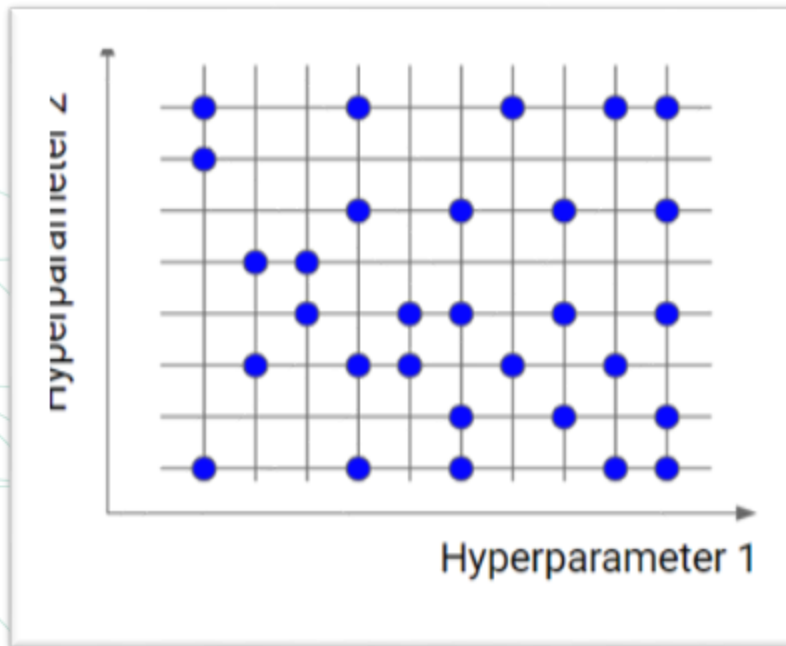
Try out all combinations of hyperparameters in a specified range.

- **Pros:** Exhaustive search through the parameter space.
- **Cons:** Computationally expensive.

Hyperparameter Tuning

Hyperparameter Tuning Techniques

Random Search



Randomly sample hyperparameter combinations from the search space.

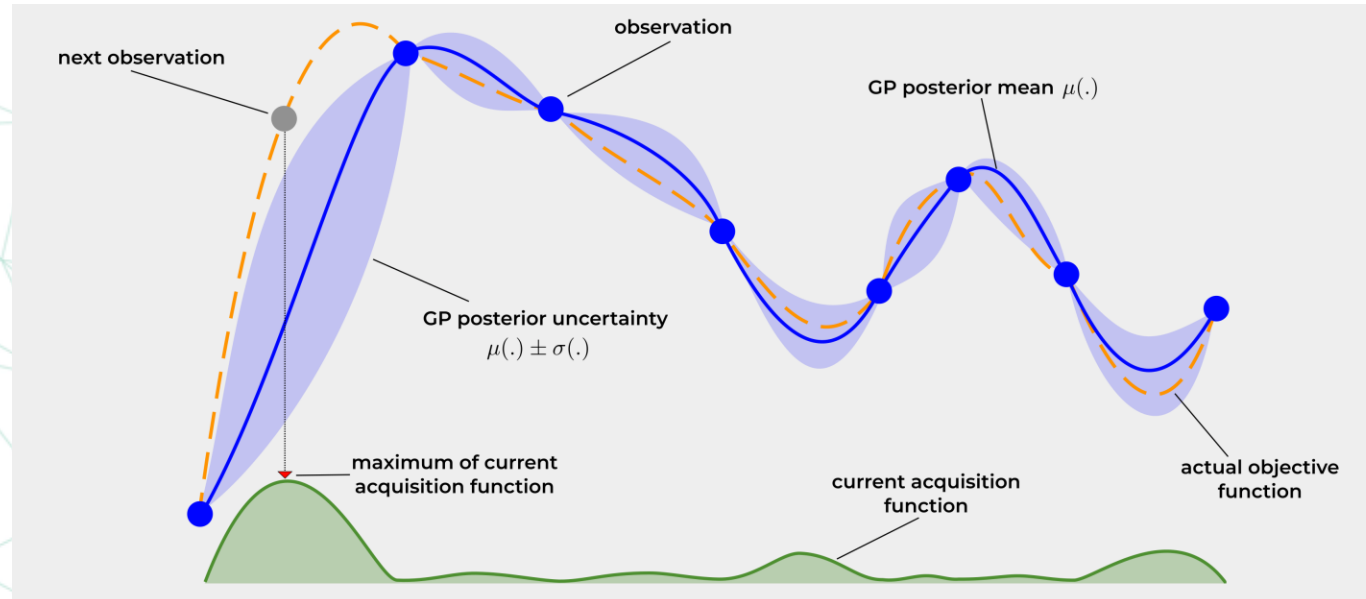
- **Pros:** Faster than grid search, can still find good combinations.
- **Cons:** May miss the best combination

Hyperparameter Tuning

Hyperparameter Tuning Techniques

Automated Hyperparameter Tuning (Bayesian Optimization)

Automatically explore the hyperparameter space intelligently to find the best settings (libraries like **Optuna** or **Keras Tuner**).

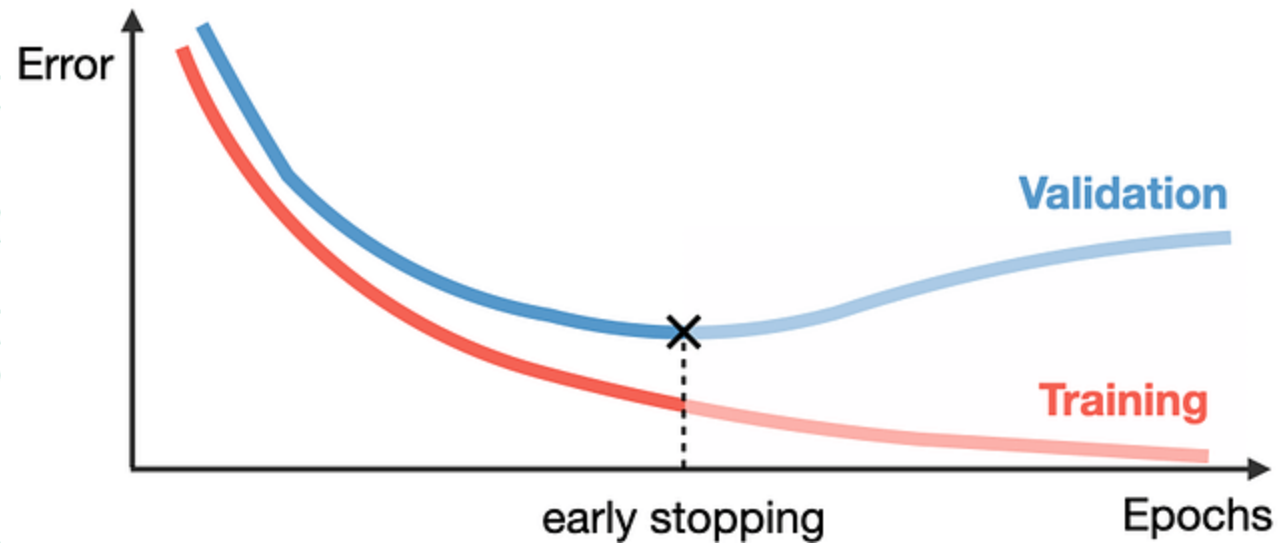


Early Stopping and Model Checkpointing

Early Stopping and Model Checkpointing

Early Stopping

- Early stopping is a regularization technique used to avoid overfitting. It monitors a performance metric (e.g., validation loss), and if the model's performance stops improving after a certain number of epochs, training is stopped.



Early Stopping and Model Checkpointing

Early Stopping

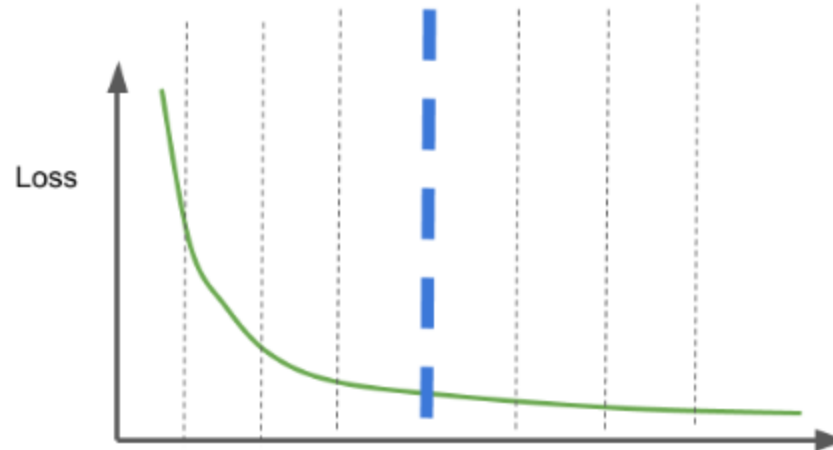
- **Patience:** The number of epochs to wait after the last improvement before stopping the training.
- **Restore Best Weights:** This option ensures that the model's weights are restored to the epoch with the best validation performance, even if training was stopped later.

Early Stopping and Model Checkpointing

Model Checkpointing

- Model checkpointing saves the model's weights after each epoch where the validation performance improves. This way, you always have the best model saved, even if training runs for many epochs and overfits later.

Retuning, starting from a checkpoint



Early Stopping and Model Checkpointing

Benefits of Early Stopping and Model Checkpointing

- **Early Stopping:** Prevents overfitting by stopping training when the validation performance no longer improves, thus avoiding unnecessary epochs.
- **Model Checkpointing:** Ensures you always have the best-performing model saved during training, even if performance deteriorates in later epochs.

Key Takeaways

1. **Transfer Learning:** Leverage pre-trained models for faster and more accurate training on specific tasks.
2. **Hyperparameter Tuning:** Optimize model performance by searching for the best hyperparameters using techniques like **Random Search** or **Grid Search**.
3. **Early Stopping and Model Checkpointing:** Prevent overfitting and save the best model during training for better generalization.